Question:

A computer store system needs to prepare different types of computers such as:

- **Gaming Computer**
- **Office Computer**
- **Server Computer**

Each computer must always be assembled using the same sequence of steps:

1. Add CPU
2. Add RAM
3. Add Storage

The components, however, differ depending on the type of computer:

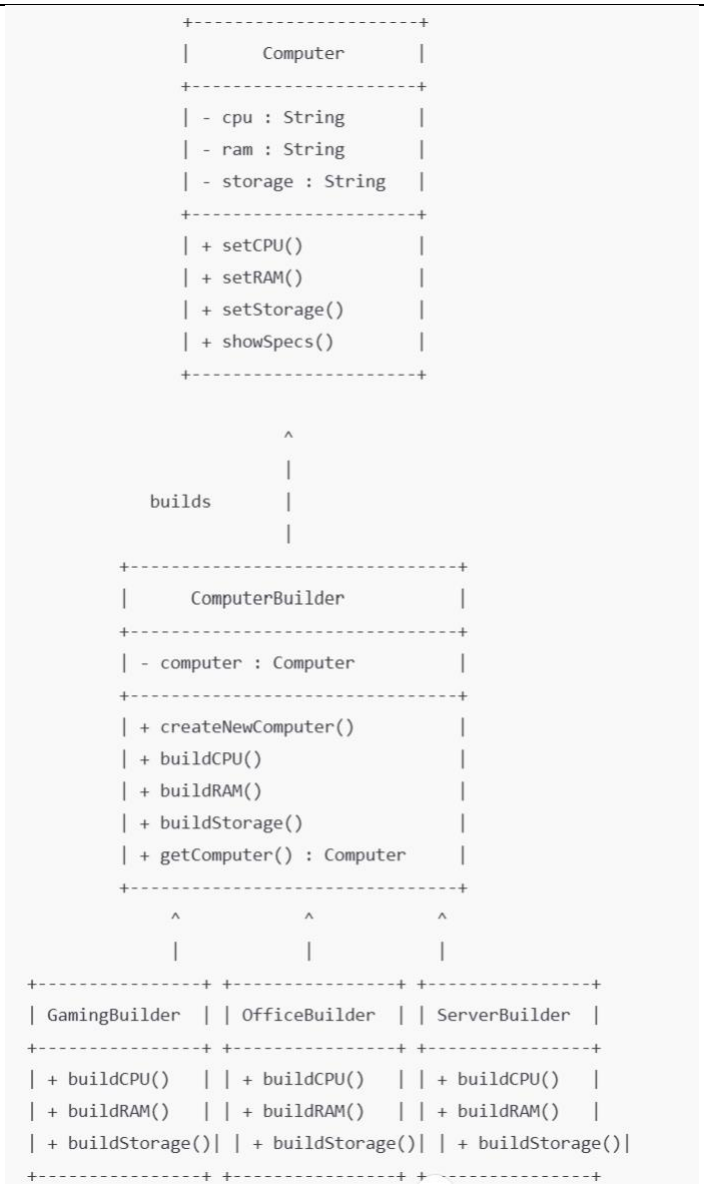
- **Gaming Computer**: Intel i9 + 32GB RAM + 1TB SSD
- **Office Computer**: Intel i5 + 16GB RAM + 512GB SSD
- **Server Computer**: AMD EPYC + 64GB RAM + 2TB NVMe

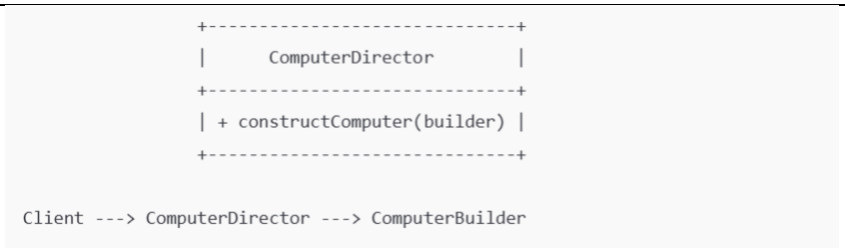
You are asked to design and implement this system in Java such that:

- The construction steps are the same for all computers.
- New types of computers can be added easily in the future.
- The client should only decide the computer type, while the actual construction process remains hidden.

Tasks:

1. Draw a class diagram based on the Builder Pattern.
2. Write the Java code.


Computer
- cpu : String
- ram : String
- storage : String
+ setCPU()
+ setRAM()
+ setStorage()
+ showSpecs()
builds
ComputerBuilder
- computer : Computer
+ createNewComputer()
+ buildCPU()
+ buildRAM()
+ buildStorage()
+ getComputer() : Computer
GamingBuilder
+ buildCPU()
+ buildRAM()
+ buildStorage()
OfficeBuilder
+ buildCPU()
+ buildRAM()
+ buildStorage()
ServerBuilder
+ buildCPU()
+ buildRAM()
+ buildStorage()


ComputerDirector
+ constructComputer(builder)
Client ---> ComputerDirector ---> ComputerBuilder

Product class

java

```java
class Computer {
    private String cpu;
    private String ram;
    private String storage;

    public void setCPU(String cpu) {
        this.cpu = cpu;
    }

    public void setRAM(String ram) {
        this.ram = ram;
    }

    public void setStorage(String storage) {
        this.storage = storage;
    }

    public void showSpecs() {
        System.out.println("Computer Specifications:");
        System.out.println("CPU: " + cpu);
        System.out.println("RAM: " + ram);
        System.out.println("Storage: " + storage);
        System.out.println();
    }
}
```

Abstract Builder

```java
abstract class ComputerBuilder {
    protected Computer computer;

    public void createNewComputer() {
        computer = new Computer();
    }

    public Computer getComputer() {
        return computer;
    }

    public abstract void buildCPU();
    public abstract void buildRAM();
    public abstract void buildStorage();
}
```

Concrete Builders

```java
class GamingComputerBuilder extends ComputerBuilder {
    @Override
    public void buildCPU() {
        computer.setCPU("Intel i9");
    }

    @Override
    public void buildRAM() {
        computer.setRAM("32GB RAM");
    }

    @Override
    public void buildStorage() {
        computer.setStorage("1TB SSD");
    }
}
```

Director class

```java
class ComputerDirector {
    public void constructComputer(ComputerBuilder builder) {
        builder.createNewComputer();
        builder.buildCPU();
        builder.buildRAM();
        builder.buildStorage();
    }
}
```

```java
class ServerComputerBuilder extends ComputerBuilder {
    @Override
    public void buildCPU() {
        computer.setCPU("AMD EPYC");
    }

    @Override
    public void buildRAM() {
        computer.setRAM("64GB RAM");
    }

    @Override
    public void buildStorage() {
        computer.setStorage("2TB NVMe");
    }
}
```

java

```java
class OfficeComputerBuilder extends ComputerBuilder {
    @Override
    public void buildCPU() {
        computer.setCPU("Intel i5");
    }

    @Override
    public void buildRAM() {
        computer.setRAM("16GB RAM");
    }

    @Override
    public void buildStorage() {
        computer.setStorage("512GB SSD");
    }
}
```


```java
class ServerComputerBuilder extends ComputerBuilder {
    @Override
    public void buildCPU() {
        computer.setCPU("AMD EPYC");
    }

    @Override
    public void buildRAM() {
        computer.setRAM("64GB RAM");
    }

    @Override
    public void buildStorage() {
        computer.setStorage("2TB NVMe");
    }
}
```


Client / Main class

java

```java
public class Main {
    public static void main(String[] args) {
        ComputerDirector director = new ComputerDirector();

        // Gaming Computer
        ComputerBuilder gamingBuilder = new GamingComputerBuilder();
        director.constructComputer(gamingBuilder);
        Computer gamingComputer = gamingBuilder.getComputer();
        System.out.println("Gaming Computer:");
        gamingComputer.showSpecs();

        // Office Computer
        ComputerBuilder officeBuilder = new OfficeComputerBuilder();
        director.constructComputer(officeBuilder);
        Computer officeComputer = officeBuilder.getComputer();
        System.out.println("Office Computer:");
        officeComputer.showSpecs();

        // Server Computer
        ComputerBuilder serverBuilder = new ServerComputerBuilder();
        director.constructComputer(serverBuilder);
        Computer serverComputer = serverBuilder.getComputer();
        System.out.println("Server Computer:");
        serverComputer.showSpecs();
    }
}
```

Expected Output

```text
Gaming Computer:
Computer Specifications:
CPU: Intel i9
RAM: 32GB RAM
Storage: 1TB SSD

Office Computer:
Computer Specifications:
CPU: Intel i5
RAM: 16GB RAM
Storage: 512GB SSD

Server Computer:
Computer Specifications:
CPU: AMD EPYC
RAM: 64GB RAM
Storage: 2TB NVMe
```